

STEROWNIKI MIKROPROCESOROWE

LINE FOLLOWER ROBOT

BARTOSZ ZAWŁOCKI

MECHATRONIKA, SEMESTR 5, NR ALBUMY 161608

Spis treści

Opis projektu	3
Obliczenia	4
Schemat blokowy	7
Schemat ideowy	8
Schemat elektryczny	14
PINOUT	15
Wykaz elementów	16
Płytką PCB	18
Program	19
Wnioski	27

Opis projektu

Założeniem projektu jest zaprojektowanie i wykonanie samodzielnie płytki PCB oraz oprogramowanie (opartego na rejestrach mikrokontrolera) dla szybkiego robota typu Line-Follower sterowanym algorytmem PD. Zadaniem robota jest poruszanie się wzdłuż czarnej linii na jasnym tle. Jako czujniki do wykrywania linii posłużą czujniki odbiciowe IR, które będą wysyłały dane o ich stanie do mikroprocesora, a następnie obliczane będzie odchylenie od linii i na tej podstawie regulator PD będzie obliczał korektę sterowania napędem. Programowanie mikroprocesora będzie odbywało się za pomocą USB typu C. Dodatkowo na płytce przewidziane jest męskie złącze typu JST dla zasilania. Robot będzie startował cały program po kliknięciu przycisku START znajdującego się w tylnej części płytki.

Obliczenia

Dioda LED

Prąd maksymalny który może przejść przez tą diodę to 15 mA. Zalecany prąd przez producenta to 2mA. Nasze napięcie V_f według noty to 1,8V.

$$R = \frac{3,3 - 1,8}{0,002} = 750 \Omega$$

W zamówionym zestawie rezystorów typu 0603 SMD mamy dwie bliskie wartości 680 Ω oraz 1k Ω . Następnie obliczymy jakie prądy będą przy danym rezystorze.

$$I_{680R} = \frac{3,3 - 1,8}{680} = 2,2 \text{ mA}$$

$$I_{1kR} = \frac{3,3 - 1,8}{1000} = 1,5 \text{ mA}$$

Najbliższa dla nas obliczona wartość jest dla rezystora 680 Ω . Teraz wyliczymy moc i czy rezystor wytrzyma.

$$P = (0,0022)^2 \cdot 680 = 3,3 \text{ mW}$$

Moc maksymalna na rezystorze 0603 to 62 mW, więc na wyżej obliczonej mocy widzimy, że nasz rezystor wytrzyma.

Dioda IR

Typowy prąd dla diody IR to około 20 mA. A napięcie przewodzenia jest równe 1,25V.

$$R = \frac{3,3 - 1,25}{0,02} = 102,5 \, \Omega$$

Najbliższy rezystor jaki możemy dobrać to 100 Ω . Następnie obliczymy jaki prąd uzyskamy dokładnie.

$$I_{100R} = \frac{3,3 - 1,25}{100} = 20,5 \, mA$$

Następnie obliczymy jaka moc wydzieli się na rezystorze.

$$P = (0,0205)^2 \cdot 100 = 42 \, mW$$

Podobnie jak w przypadku z diodą LED nasza wartość jest mniejsza niż maksymalna moc dla rezystorów 0603.

Prędkość pojazdu

Koła wymodelowane w programie 3D typu CAD mają średnice 37 mm. Prędkość obrotowa silnika prądu stałego to 1000 obr/min. Mając te dane możemy wyliczyć teoretyczną prędkość. Najpierw wyliczamy obwód koła.

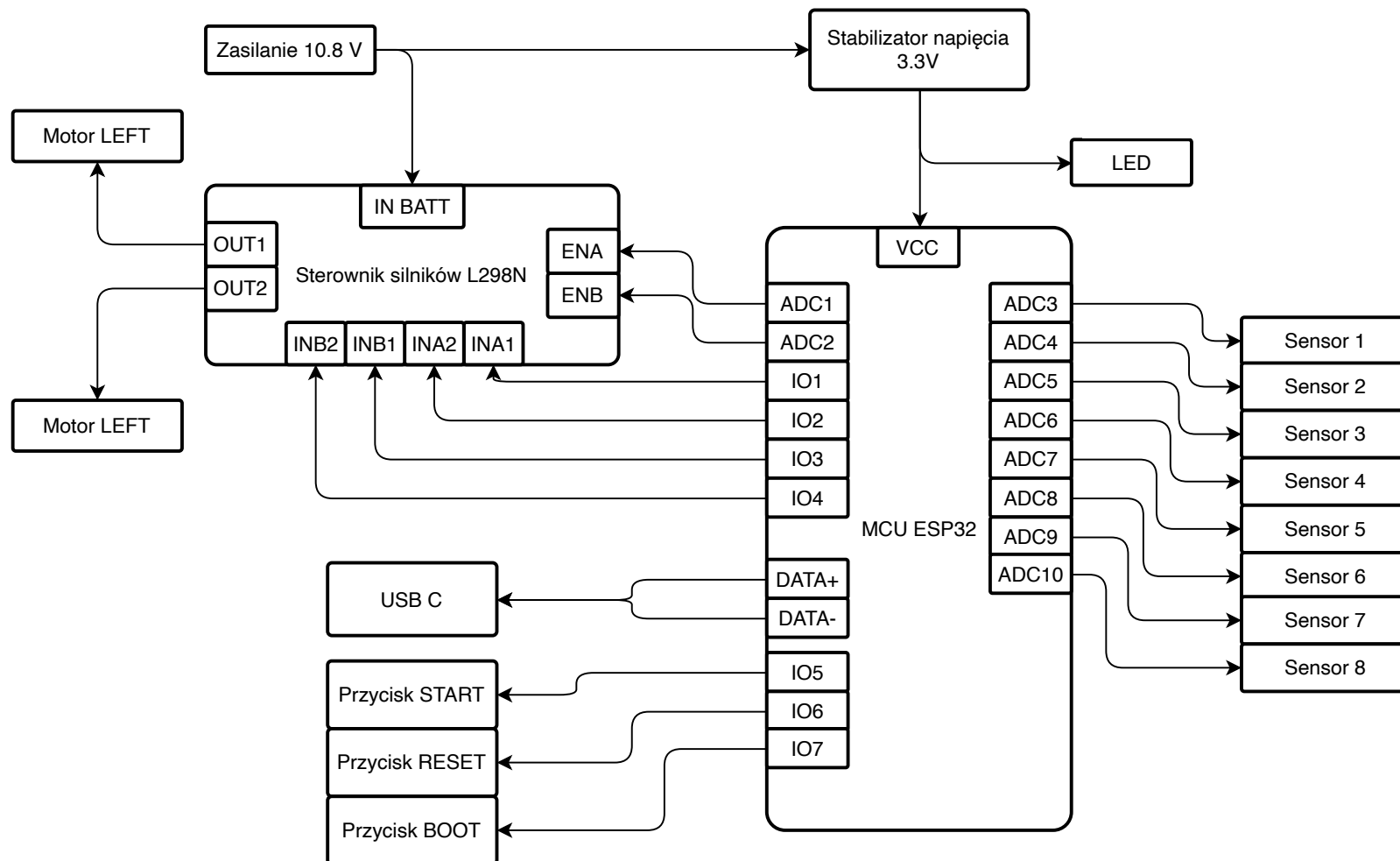
$$C = \pi \cdot 0,037 = 0,11624 \text{ m}$$

Mając obwód możemy już podstawić wszystko do wzoru na prędkość.

$$v = \frac{1000}{60} \cdot 0,116239 = 1,94 \frac{\text{m}}{\text{s}}$$

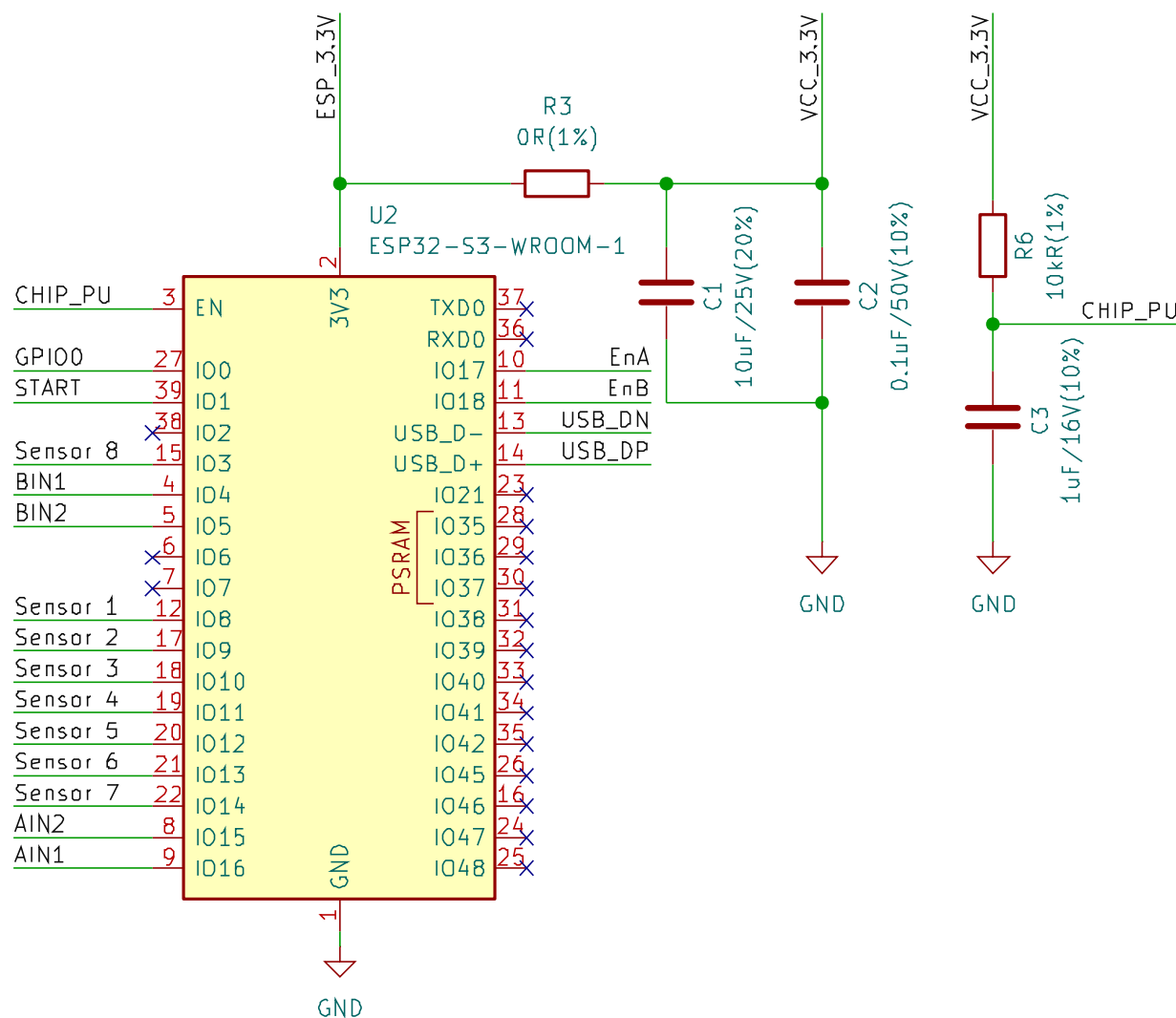
Obliczona prędkość w zupełności wystarczy. W sieci możemy znaleźć, że standardowa prędkość jest od 1 do 2 m/s, a najszybsze osiągają nawet 3 m/s. Jeżeli nasza prędkość będzie za wysoka możemy modulować prędkość silnika za pomocą sygnału PWM na piny sterownika L298n.

Schemat blokowy

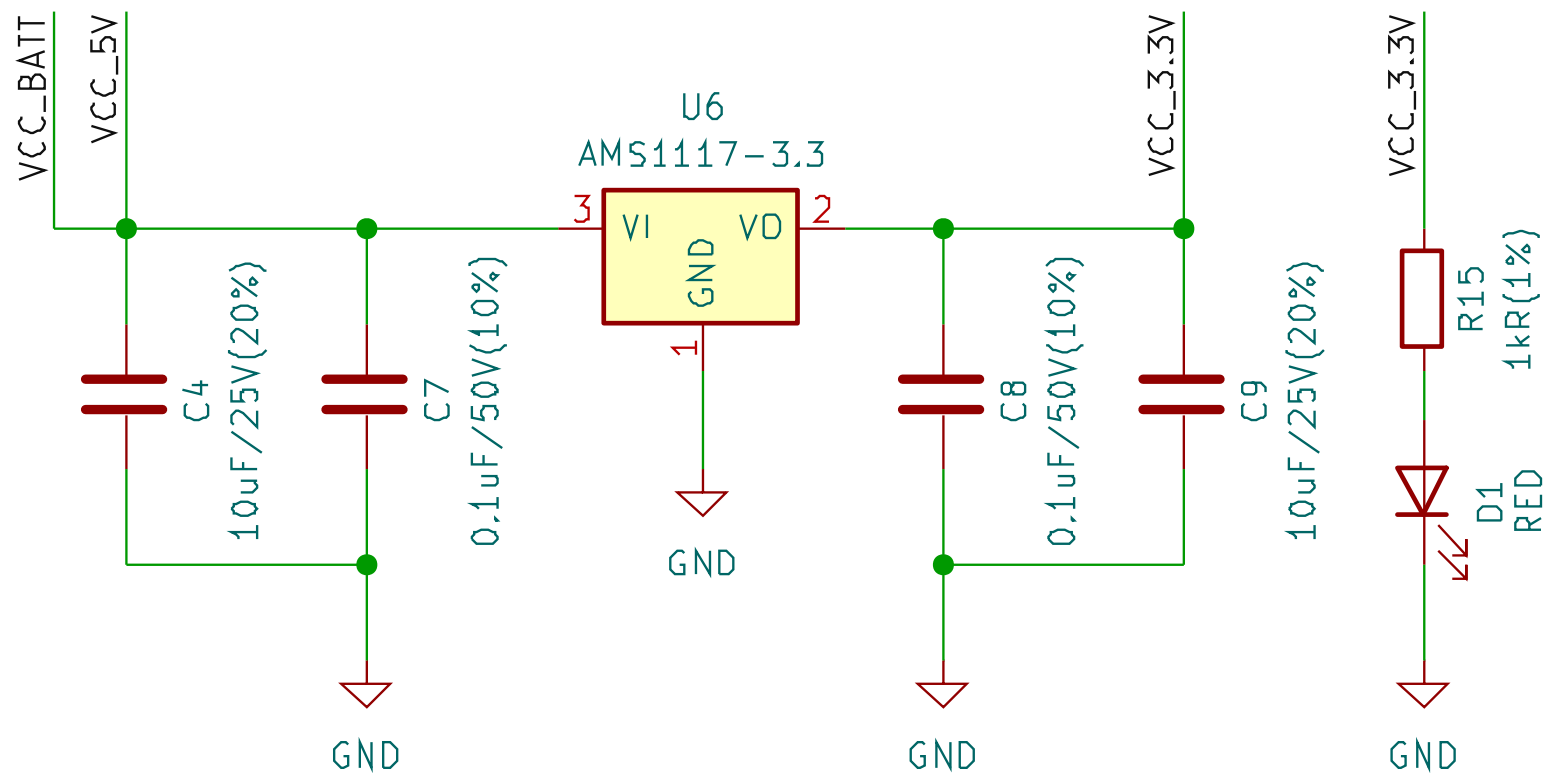


Schemat ideowy

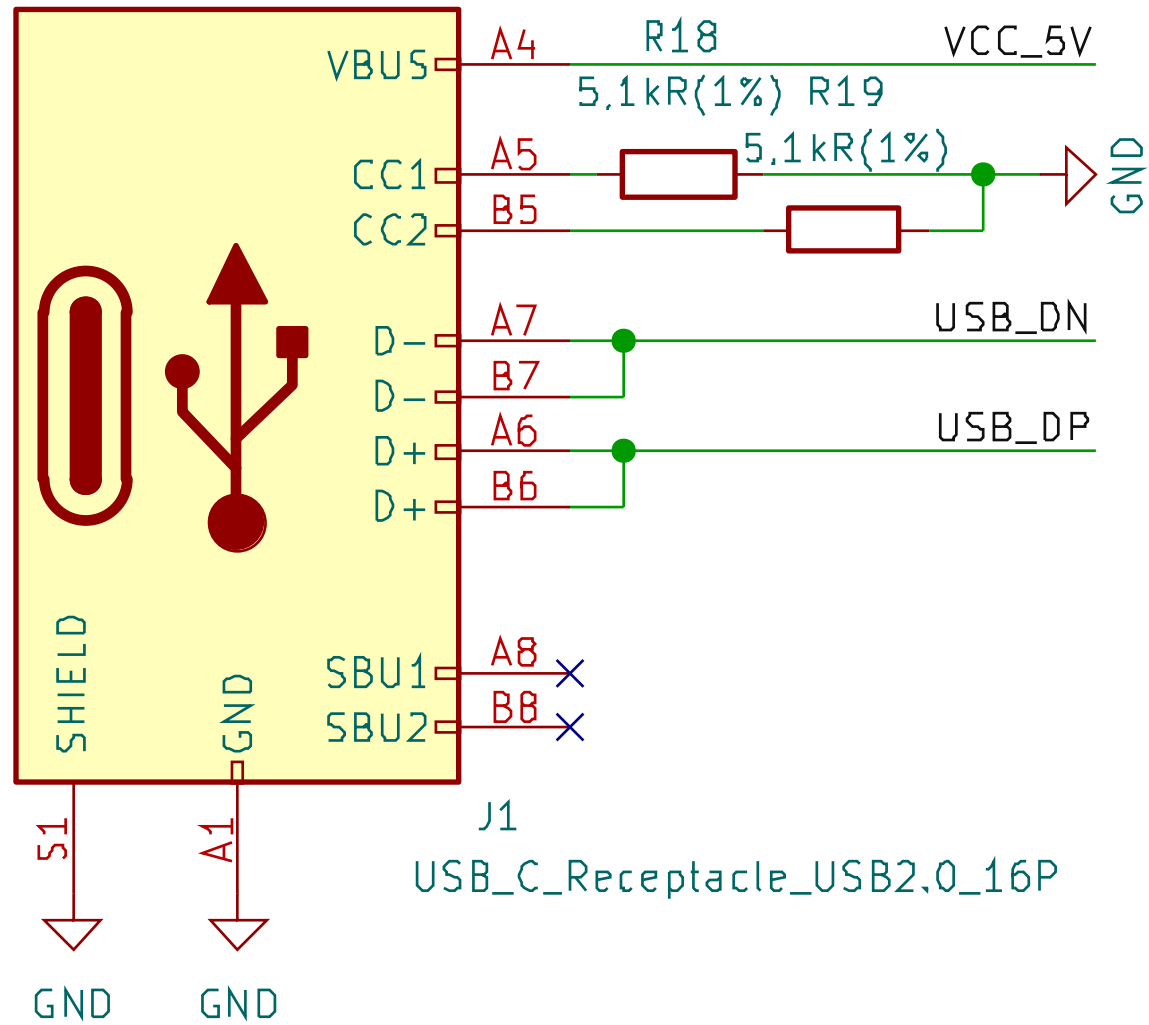
MCU



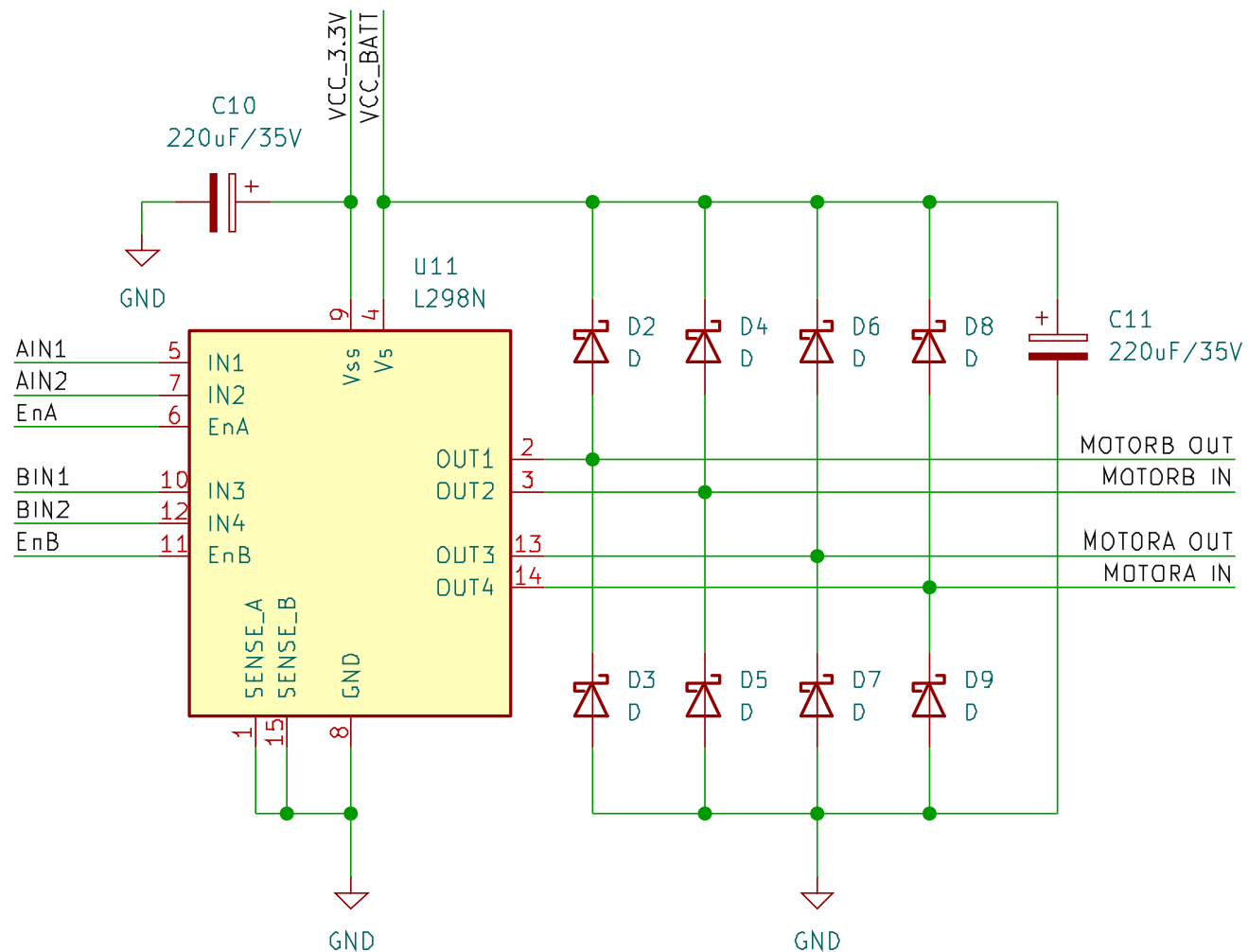
Stabilizator napięcia 3.3V



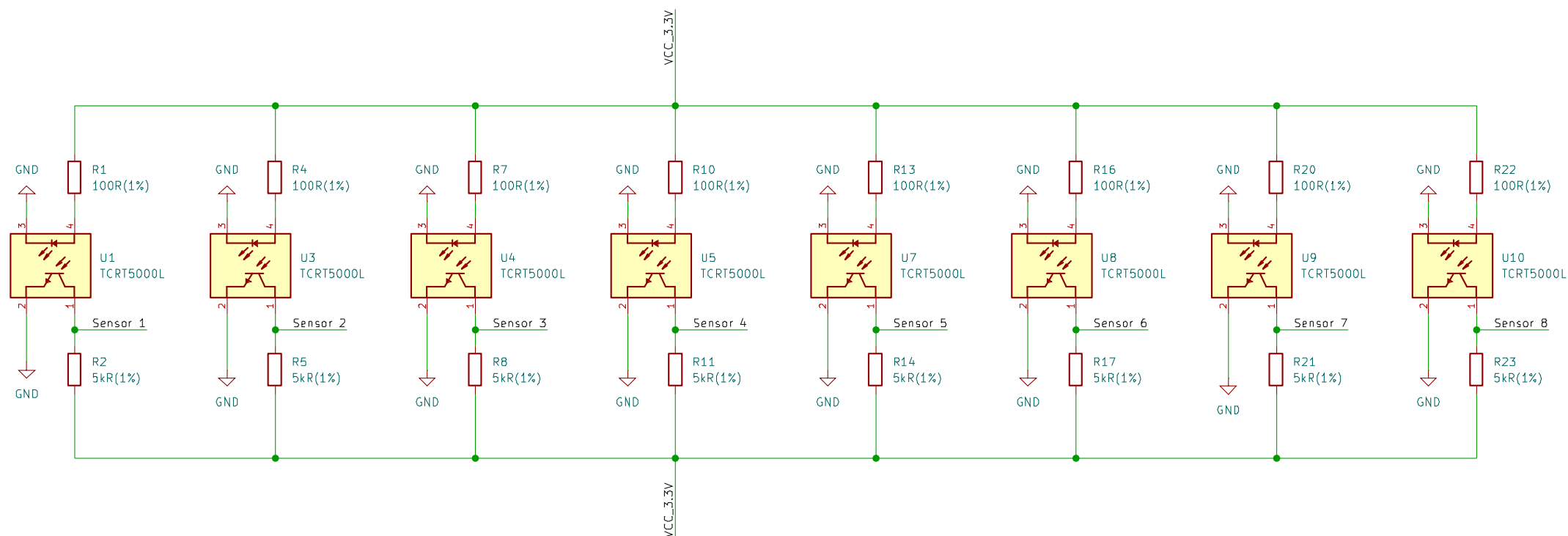
USB typu C



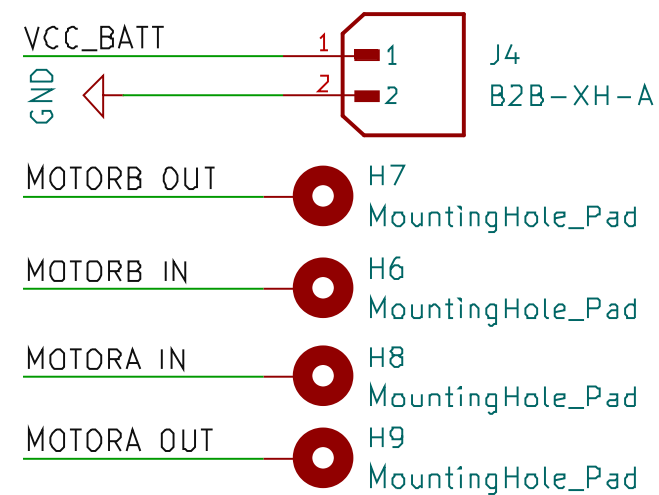
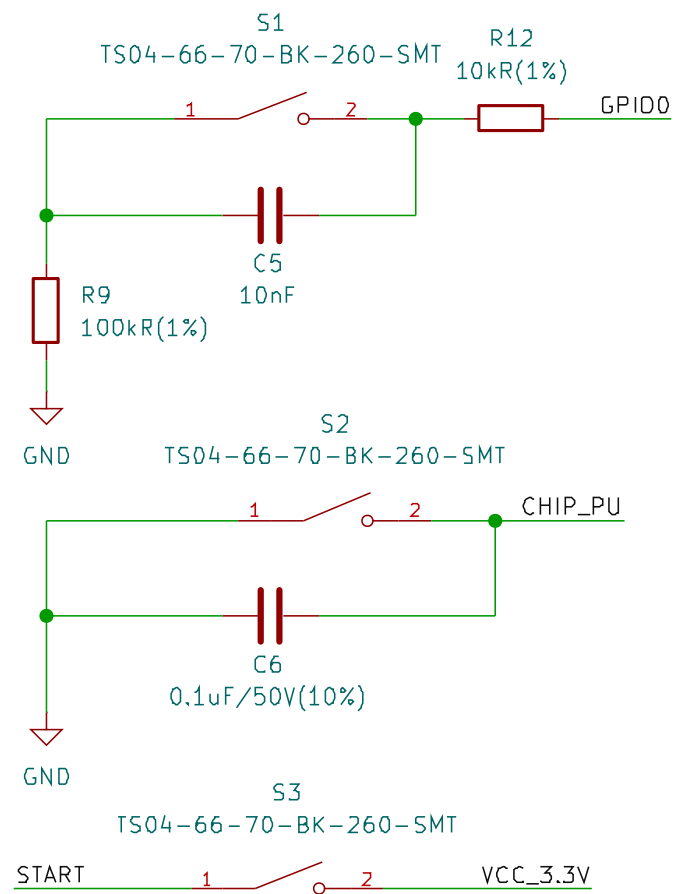
Sterownik silników



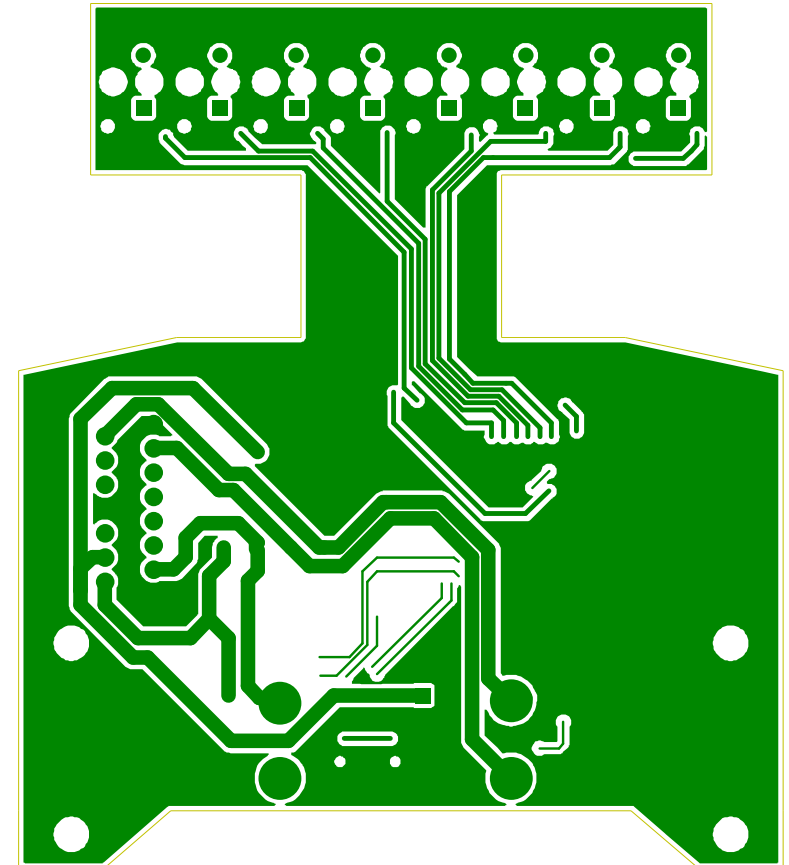
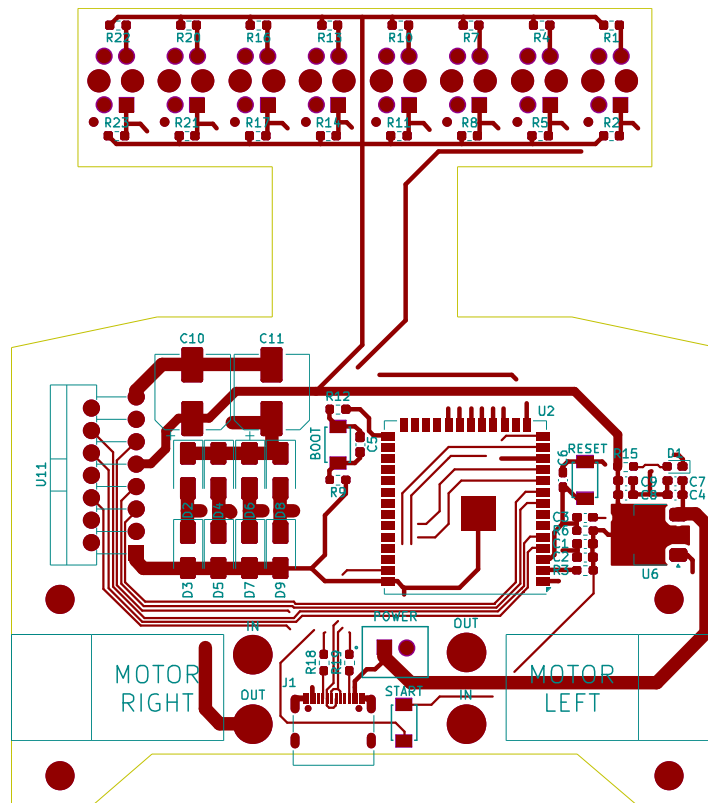
Sensory odbiciowe



Przyciski oraz złącza



Schemat elektryczny



PINOUT

Lp	Oznaczenie na schemacie	Funkcja
1	EN	Przycisk reset
2	IO0	Przycisk boot
3	IO1	Przycisk start
4	IO3	Sensor nr 8
5	IO4	Wejście B1 sterownika silnika
6	IO5	Wejście B2 sterownika silnika
7	IO8	Sensor nr 1
8	IO9	Sensor nr 2
9	IO10	Sensor nr 3
10	IO11	Sensor nr 4
11	IO12	Sensor nr 5
12	IO13	Sensor nr 6
13	IO14	Sensor nr 7
14	IO15	Wejście A2 sterownika silnika
15	IO16	Wejście A1 sterownika silnika
16	IO17	Wejście enable A sterownika silnika
17	IO18	Wejście enable B sterownika silnika
18	IO19	USB DATA -

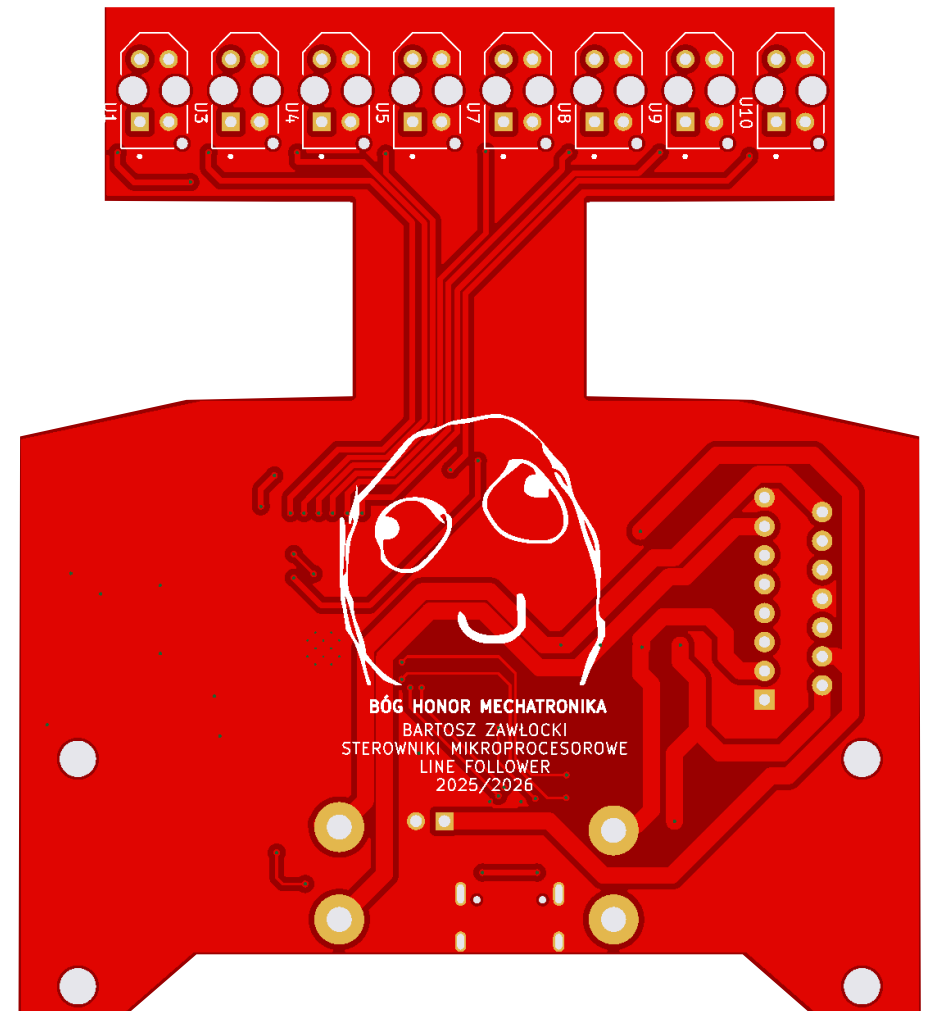
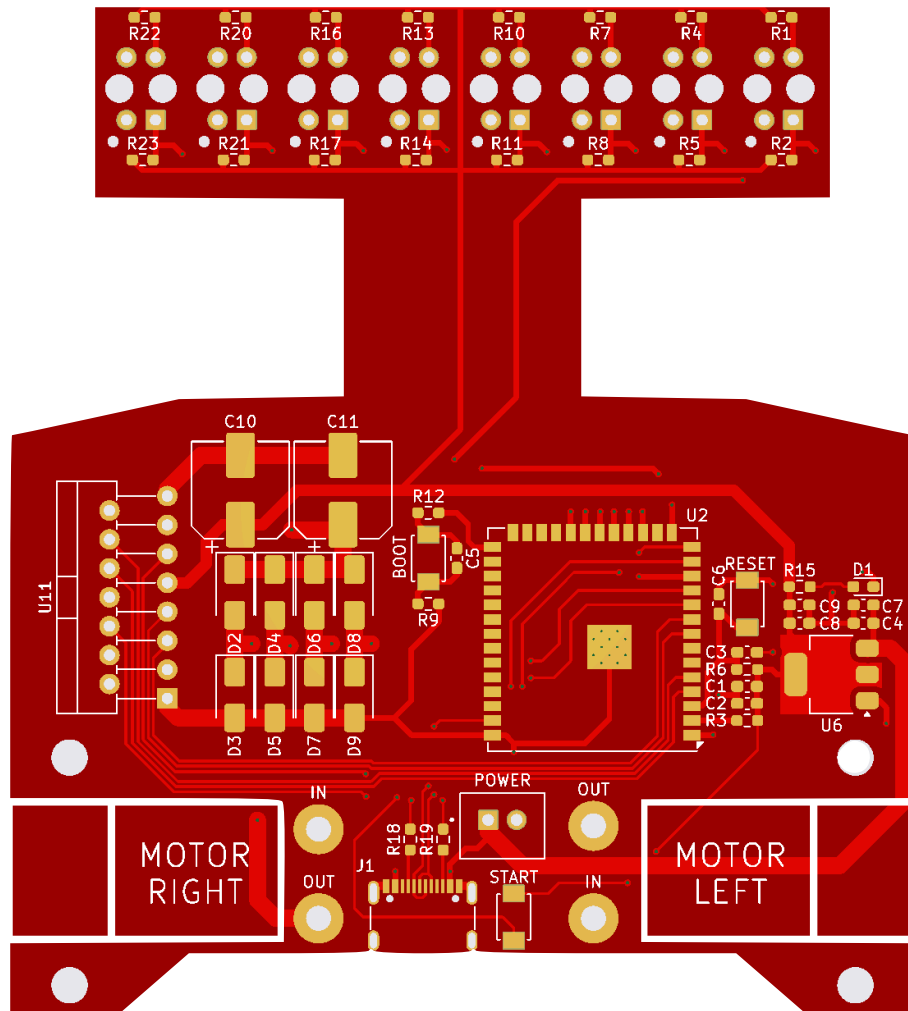
Lp	Oznaczenie na schemacie	Funkcja
19	IO20	USB DATA +

Wykaz elementów

Lp	Oznaczenie	Nazwa	Wartość	Ilość
1	C2, C6, C7, C8	Kondensator 0603	0.1 uF	4
2	C1, C4, C9	Kondensator 0603	10 uF	3
3	C10, C11	Kondensator 8x6,2	220 uF	2
4	C3	Kondensator 0603	1 uF	1
5	C5	Kondensator 0603	10 nF	1
6	R1, R4, R7, R10, R13, R16, R20, R22	Rezystor 0603	100 R	8
7	R2, R5, R8, R11, R14, R17, R21, R23	Rezystor 0603	5k R	8
8	R6, R12	Rezystor 0603	10k R	2
9	R18, R19	Rezystor 0603	5,1k R	2
10	R3	Rezystor 0603	0 R	1
11	R9	Rezystor 0603	100k R	1
12	R15	Rezystor 0603	1k R	1
13	D2, D3, D4, D5, D6, D7, D8, D9	Dioda Schottky	SS32 SMA	8
14	D1	Dioda led	RED	1
15	U1, U3, U4, U5, U7, U8, U9, U10	Dioda IR i fototranzystor	TCRT5000L	8

Lp	Oznaczenie	Nazwa	Wartość	Ilość
16	U2	Mikroprocesor	ESP32-S3-WROOM-1U	1
17	U6	Stabilizator napięcia 3.3V	AMS1117-3.3	1
18	U11	Sterownik silników	L298N	1
19	S1, S2, S3	Przycisk	TS-1286	3
20	J2	Złącze zasilania	JST-XH-A	1
21	J1	Złącze programowania	USB C 16 pin	1

Płytki PCB



Program

```
#include <Arduino.h>
#include "driver/ledc.h"
#include "driver/gpio.h"
#include "driver/adc.h"
#include "soc/gpio_struct.h"
#include „esp_timer.h”
```

Zastosowane biblioteki w programie

```
#define USTAW_PIN(pin) (GPIO.out_w1ts = (1UL << pin))
#define ZERUJ_PIN(pin) (GPIO.out_w1tc = (1UL << pin))
```

Zdefiniowane makro do szybkiego ustawiania pinów HIGH/LOW na rejestrach

```
#define PIN_PRZYCISK 1
#define AIN1 16
#define AIN2 15
#define ENA 17
#define BIN1 4
#define BIN2 5
#define ENB 18
```

Zdefiniowane pinów sterownika mostka H oraz przycisku START

```
#define CZESTOTLIWOSC_PWM 20000
#define ROZDZIELCZOSC_PWM 8
#define KANAL_A LEDC_CHANNEL_0
#define KANAL_B LEDC_CHANNEL_1
#define TIMER_PWM LEDC_TIMER_0
```

Ustawienie PWM dla sterowania mostkiem H

```
#define PROG_CZARNY 1500
```

Wartość dla wykrywania czarnej linii

```
#define CZAS_BOOST 250
```

Czas przyspieszania podczas startu

```
#define PREDKOSC_BAZOWA 200  
#define PREDKOSC_MIN 150  
#define PREDKOSC_MAX 255
```

Ustawienie prędkości dla silnika

```
#define PROG_BRAK_LINII 400
```

Próg do uznania braku czarnej linii

```
#define CZAS_BRAK_LINII 3000
```

Czas zatrzymania robota przy braku wykrycia linii w ms

```
const float Kp = 1.8;  
const float Kd = 0.6;
```

Wartości członu proporcjonalnego i różniczkowego

```
int wartosci_czujnikow[8];
```

Tablica do przechowywania odczytów z 8 sensorów

```
int ostatni_blad = 0;
```

Pamięć błędu z poprzedniej iteracji PD

```
bool start = false;
```

Start robota

```
uint64_t licznik_brak_linii = 0;
```

Timer braku linii

```

void inicjalizacjaGPIO(){
    gpio_config_t io = {};
    io.mode = GPIO_MODE_OUTPUT;
    io.pin_bit_mask =
        (1ULL<<AIN1) |
        (1ULL<<AIN2) |
        (1ULL<<BIN1) |
        (1ULL<<BIN2);
    gpio_config(&io);

    ZERUJ_PIN(AIN1);
    ZERUJ_PIN(AIN2);
    ZERUJ_PIN(BIN1);
    ZERUJ_PIN(BIN2);

    io.mode = GPIO_MODE_INPUT;
    io.pin_bit_mask = (1ULL<<PIN_PRZYCISK);
    gpio_config(&io);
}

```

Inicjalizacja GPIO, gdzie konfigurowane są piny mostka H jako wyjścia, ustawiamy im wartość LOW na start oraz ustawienie GPIO przycisku START jako wejście.

```

void inicjalizacjaPWM(){
    ledc_timer_config_t timer = {};
    timer.speed_mode = LEDC_LOW_SPEED_MODE;
    timer.duty_resolution = (ledc_timer_bit_t)ROZDZIELCZOSC_PWM;
    timer.timer_num = TIMER_PWM;
    timer.freq_hz = CZESTOTLIWOSC_PWM;
    timer.clk_cfg = LEDC_AUTO_CLK;
    ledc_timer_config(&timer);

    ledc_channel_config_t kanalA = {};
    kanalA.gpio_num = ENA;
    kanalA.speed_mode = LEDC_LOW_SPEED_MODE;
    kanalA.channel = KANAL_A;
    kanalA.timer_sel = TIMER_PWM;
    kanalA.duty = 0;
    kanalA.hpoint = 0;
    ledc_channel_config(&kanalA);

    ledc_channel_config_t kanalB = kanalA; // kopiujemy konfiguracje
    kanalB.channel = KANAL_B;
    kanalB.gpio_num = ENB;
    ledc_channel_config(&kanalB);

    ledc_set_duty(LEDC_LOW_SPEED_MODE, KANAL_A, 0);
    ledc_update_duty(LEDC_LOW_SPEED_MODE, KANAL_A);
    ledc_set_duty(LEDC_LOW_SPEED_MODE, KANAL_B, 0);
    ledc_update_duty(LEDC_LOW_SPEED_MODE, KANAL_B);
}

```

Konfiguracja timera PWM dla mostka H. Dla dwóch kanałów konfigurację są bliźniacze. Na sam koniec ustawiamy początkowe wypełnienie równe 0%.

```
inline void ustawPredkosc(uint8_t predkoscA, uint8_t predkoscB){
    ledc_set_duty(LED_C_LOW_SPEED_MODE, KANAL_A, predkoscA);
    ledc_update_duty(LED_C_LOW_SPEED_MODE, KANAL_A);
    ledc_set_duty(LED_C_LOW_SPEED_MODE, KANAL_B, predkoscB);
    ledc_update_duty(LED_C_LOW_SPEED_MODE, KANAL_B);
}
```

Ustawianie prędkości dla kanału A oraz B mostka H.

```
void kickStart(){
    ustawPredkosc(255,255);
    delay(CZAS_BOOST);
    ustawPredkosc(PREDKOSC_BAZOWA, PREDKOSC_BAZOWA);
}
```

Ustawienie wspomagania startu poprzez ustawienie maksymalnej wartości na silniki przez określony czas.

```
inline void jazdaProsto(int a, int b){
    USTAW_PIN(AIN1); USTAW_PIN(BIN1);
    ZERUJ_PIN(AIN2); ZERUJ_PIN(BIN2);
    ustawPredkosc(a,b);
}
```

Ustawienie jazdy prosto.

```
inline void zatrzymajSilniki(){
    ustawPredkosc(0,0);
    ZERUJ_PIN(AIN1); ZERUJ_PIN(AIN2); ZERUJ_PIN(BIN1); ZERUJ_PIN(BIN2);
}
```

Zatrzymanie silników.

```
inline void skretPrawo90(){
    USTAW_PIN(AIN1); USTAW_PIN(BIN2);
    ZERUJ_PIN(AIN2); ZERUJ_PIN(BIN1);
    ustawPredkosc(200,200);
}
```

Skręt w prawo 90 stopni.

```
inline void skretLewo90(){
    USTAW_PIN(AIN2); USTAW_PIN(BIN1);
    ZERUJ_PIN(AIN1); ZERUJ_PIN(BIN2);
    ustawPredkosc(200,200);
}
```

Skręt w lewo 90 stopni.

```
void odczytajCzujniki(){
    int wartosc;

    wartosci_czujnikow[0] = adc1_get_raw(ADC1_CHANNEL_7); // GPIO8
    wartosci_czujnikow[1] = adc1_get_raw(ADC1_CHANNEL_8); // GPIO9
    wartosci_czujnikow[2] = adc1_get_raw(ADC1_CHANNEL_9); // GPIO10
    wartosci_czujnikow[7] = adc1_get_raw(ADC1_CHANNEL_2); // GPIO3

    adc2_get_raw(ADC2_CHANNEL_0, ADC_WIDTH_BIT_12, &wartosc); wartosci_czujnikow[3] = wartosc;
    adc2_get_raw(ADC2_CHANNEL_1, ADC_WIDTH_BIT_12, &wartosc); wartosci_czujnikow[4] = wartosc;
    adc2_get_raw(ADC2_CHANNEL_2, ADC_WIDTH_BIT_12, &wartosc); wartosci_czujnikow[5] = wartosc;
    adc2_get_raw(ADC2_CHANNEL_3, ADC_WIDTH_BIT_12, &wartosc); wartosci_czujnikow[6] = wartosc;
}
```

Odczyt z czujników poprzez ADC1 oraz ADC2 i zapisanie wartości do tablicy

```
int policzBlad(){
    int wagi[8] = {-3000,-2000,-1000,-300,300,1000,2000,3000};
    long suma = 0;
    long suma_wazona = 0;
    for(int i = 0; i < 8 ; i++){
        int val = wartosci_czujnikow[i] > PROG_CZARNY ? wartosci_czujnikow[i] : 0;
        suma += val * wagi[i];
        suma_wazona += val;
    }
    if (suma_wazona == 0) return 0;
    return suma/suma_wazona;
}
```

Nadanie wagi dla poszczególnych sensorów oraz obliczenie błędu.

```
void setup(){
```

```

Serial.begin(115200);
inicjalizacjaGPIO();
inicjalizacjaPWM();
zatrzymajSilniki();
Serial.println("Kliknij przycisk");
}

```

Pętla setUp w której wywołujemy wcześniej stworzone funkcje do przypisania funkcji odpowiednim rejestrom.

```

void loop(){
    if(!start){
        if(GPIO.in & (1<<PIN_PRZYCISK)){
            delay(50);
            if(GPIO.in & (1<<PIN_PRZYCISK)){
                Serial.println("Start za 3 sekundy...");
                delay(3000);
                start = true;
                kickStart();
            }
        }
        return;
    }

    odczytajCzujniki();

    bool linia = false;
    for(int i = 0 ; i < 8 ; i++){
        if(wartosci_czujnikow[i] > PROG_BRAK_LINII){
            linia = true;
            break;
        }
    }

    uint64_t teraz = esp_timer_get_time()/1000;

    if(!linia){
        if(licznik_brak_linii==0) licznik_brak_linii = teraz;
        else if(teraz-licznik_brak_linii >= CZAS_BRAK_LINII){
            zatrzymajSilniki();
            start = false;
            licznik_brak_linii = 0;
            return;
        }
    }
}

```

```

    }
} else licznik_brak_linii = 0;

bool srodekLinia = wartosci_czujnikow[3] > PROG_CZARNY || wartosci_czujnikow[4] > PROG_CZARNY;
bool srodekBialy = wartosci_czujnikow[3] <= PROG_CZARNY && wartosci_czujnikow[4] <= PROG_CZARNY;

if(wartosci_czujnikow[0] > PROG_CZARNY && srodekBialy){ skretPrawo90(); return; }
if(wartosci_czujnikow[7] > PROG_CZARNY && srodekBialy){ skretLewo90(); return; }
if(srodekLinia){ jazdaProsto(PREDKOSC_BAZOWA, PREDKOSC_BAZOWA); return; }

int blad = policzBlad();
int pochodna = blad - ostatni_blad;
ostatni_blad = blad;

int korekta = Kp*blad + Kd*pochodna;
int predkosc_lewa = constrain(PREDKOSC_BAZOWA + korekta, PREDKOSC_MIN, PREDKOSC_MAX);
int predkosc_prawa = constrain(PREDKOSC_BAZOWA - korekta, PREDKOSC_MIN, PREDKOSC_MAX);

ustawPredkosc(predkosc_prawa, predkosc_lewa);
}

```

Główna pętla loop która odpowiada pętli while podczas programowania na rejestrach w architekturze Atmegi. Na samym początku czeka na przyciśnięcie przycisku START. Po 3 sekundach rozpoczyna się program, w pierwszej kolejności odczytuje wartości z czujników, a następnie na podstawie warunku sprawdza czy jest linia jeżeli nie włącza się timer który po 3s wyłączy silniki. Następnie sprawdzamy czy jeden z dwóch środkowych czujników wykrył czarną linię. Następnie mamy wartości z skrajnych czujników, które określają nam ostre skrety o 90 stopni i od razu uruchamiamy funkcję w zależności od czujnika skretPrawo90 lub skretLewo90. Jeżeli linia jest na środku z ustawioną prędkością robót jedzie prosto. Następnie dokonujemy korektę prędkości silników za pomocą członów PD, i ustawiamy prędkości dla lewej i prawej osi.

Wnioski

Po zakończeniu projektu zauważyłem kilka aspektów wymagających poprawy. Nie uwzględniłem złącza awaryjnego do komunikacji z zewnętrznym modulem UART, co wymusiło bezpośrednie podłączenie się do płytki.

Rozmieszczenie i liczba czujników również wymagałaby korekty. Obecny rozstaw wynoszący około 80 mm powodował, że robot czasami gubił linię na ostrych zakrętach. Tymczasowo problem rozwiązano w taki sposób, że przy wykryciu ostrzejszego zakrętu jedno koło cofało się, a drugie poruszało się prosto. W przyszłości zastosowanie nieparzystej liczby czujników pozwoliłoby na bardziej precyzyjne pozycjonowanie względem linii, zamiast obecnego systemu działającego na zasadzie bramki OR.

Rozkład elementów na płytce PCB można by zoptymalizować poprzez przesunięcie cięższych komponentów, takich jak mostek H, do tylnej części płytki, co poprawiłoby stabilność i rozkład masy robota.

Koła wykonane w technologii druku 3D miały zbyt niski współczynnik tarcia, co uniemożliwiało skuteczne poruszanie się robota. Tymczasowym rozwiązaniem było owinięcie kół taśmą izolacyjną (stroną klejącą na zewnątrz). W przyszłości można rozważyć zastosowanie osłon z odlewu silikonu, co zwiększyłoby przyczepność.

Po zmianach związanych z kołami pojawił się problem dociążenia robota. Testy różnych ustawień pakietu akumulatorów 18650 wykazały, że optymalne działanie uzyskuje się, gdy akumulatory znajdują się w tylnej części obudowy. Robot czasami ma również trudności z ruszeniem po naciśnięciu przycisku START, dlatego kolejnym krokiem powinno być zastosowanie silników o większym momencie obrotowym.